

The 6th International Conference on Multi-modal Information Analytics

Distributed Fault Root Cause Localization and Self-Healing Strategy Generation Based on Causal Inference

Xiao Ma*

Cloud Data Technologies, eBay, San Jose, 95125, California, United States

Abstract

Correlation-based RCA becomes susceptible to cascading failures in distributed cloud-native systems. Methods: As part of our proposal, we suggest CIRCA-SH, a causal reasoning-based closed-loop system that: (i) identifies anomalies based on metrics/tracks/logs, (ii) acquires a service-level causal structure model using intervention, (iii) orders the root causes according to their counterfactual probability and (iv) devises safe self-healing measures by evaluating counterfactuals with risk restrictions. In RCAEval data (sock store, online boutique, train tickets), CIRCA-SH performs better in Precision@3 by 7-24% over a robust baseline, decreasing the average recovery time by 18-31 times which is statistically meaningful ($p < 0.01$, 95 percent confidence interval). The causal explanations facilitate interpretability of localization, and possible remedial actions, which will be addressed further as we explore scalability constraints and future online causal modifications.

© 2026 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0>)

Peer-review under responsibility of the scientific committee of MMIA conference Program Committee

Keywords: Causal reasoning, root cause analysis, microservices, self-healing, counterfactual reasoning

1. Introduction

Cloud-native and microservice architectures split applications into many independent components and produce intricate interdependencies by using network access, message queues, and shared middleware. It has advantages of scaling in a flexible manner and deploying in an agile way, but it also has the issue of: rapid fault propagation, various expressions of the faults, and high signal-to-noise ratio of observable data: a resource oscillation, a flow control strategy error, or even a lower-level dependency degradation can be transformed into a timeout storm, retry flood, circuit breakers and cascading errors in seconds through the call network. In practice, in production, they nearly always include metrics, traces, and logs, and have shared origins (e.g., network oscillations), and feedback cycles (e.g., retries result in increased loads). In these situations, when root cause analysis of AIOps relies merely on

* Corresponding author. Tel.: +0-000-000-0000 ; fax: +0-000-000-0000 .

E-mail address: ella97.work@gmail.com

correlation or a fixed dependency diagram, it is prone to generating false correlations and that the anomaly nearest to the root cause is the innocent party, which results in mismanagement and the swapping of versions and so delays the time to reach service level objectives [1-3].

According to a recent research, causal reasoning offers a more interpretable and transferable modeling viewpoint of distributed fault diagnosis since it states explicitly the causal relationship between the term service-metric event that it uses counterfactual reasoning to respond to the important question of operation, i.e., Will measures taken on some particular service remedy the issue? [2,4]. Causal models can locate (i) causal transmission points, (ii) simultaneous fault points produced by the same cause, and (iii) noise interference points with no causal relationship compared with correlation analysis, which is closer to the possible outcomes expected in engineering.

Nevertheless, there remain three classes of problems that should be addressed in the engineering realization of causal algorithms: (1) The observation of high-dimensional heterogeneous information combined with the presence of multi-level complex causes presents challenges such as the complexity of structure learning and extensive inference computation; (2) Dependency graphs will change with the change of the versions of the software or the traffic conditions, and it is hard to find a single fixed causal dependency graph to solve the generalization performance problem under cross-operation conditions [5 - 6]; (3) Almost all the existing work is capable of diagnosing, but cannot self-heal. Although a root cause priority list can be produced, there is still a missing step in building automated self-healing schemes with closed loops and safety constraint check.

With this in mind, the current paper has constructed the CIRCA-SH framework that will achieve both root cause Top-K search and self-healing strategy outputs on a structured causal graph. The paper aims at training an interpretable SCM and representing the self-healing behavior as intervention measures using observable samples and controllable experiment without large scale annotations, and finding the best execution list depending on counterfactual impact evaluation.

2. Literature Review

In connection with the topic of the current paper, which is Distributed Root Cause Localization and Automated Repair Mechanism Generation Based on Causal Inference, current developments in similar work demonstrate that it has shifted away the conventional approaches of alarm correlation detectors in favour of a closed-loop intelligent operation system that combines machine learning, causal inference, and automated operations (AIOps). Veeravalli believes that self-healing distributed systems are the key orientation of the future development of AI systems able to resist structural load; it is mostly done by controlling the interconnection of the three major steps, namely sensing, diagnosis, and self-recovery, which allows reducing the threat of cascading failures in the complex distributed systems [1]. Expanding on this basis, Rawas et al. presented SystemLens, a system which incorporates performance prediction, anomaly detection, and cause identification into an integrated system, thus representing the fault detection development in a unimodal paradigm towards a multimodal paradigm involving the interplay between several modes [2]. An AI-based root cause analysis engine that is intended to analyze deep, multi-layered systems independently was developed by Tutuncuoglu, which helps reduce detection time and improve the efficiency of responses [3]. Yi et al. created CloudHeal, which uses some light weight online learning methods to provide self-healing capabilities to cloud-native systems and indicates that online iteration and dynamism are the main principles of automated repair engineering [4]. Also, it is shown in this work that the self-healing property is not only dependent on how quickly faults can be located but instead is based on the ability of models to respond to changing conditions in the operating environment [5]. The ML-heal system, developed by Al-Naameh, comprises both proactive fault detection and automatic recovery, showing that machine learning-driven predictive maintenance may significantly impact the predictiveness of repair operations and the overall performance of systems in the cloud [6]. In the fields of electric power and large-scale industrial operations, Darshan et al. have used the large language model (LLM) together with adaptive optimization algorithms to enable the micro-grid healing process through self-healing, which shows how generative intelligence can push the limits of the real-time decision-making abilities of traditional fault recovery systems [7]. In terms of reliability, Rajwalasimha et al. came up with a framework of cyber-physical systems based on a federated learning-based self-healing paradigm; they asserted that the future self-healing systems should be able to not only achieve self-repair but also have the ability to be interpretable, transferable and collaboratively intelligent [8]. Last but not least, Somayajula offered a hybrid AIOps methodology

which encompasses three different ways - statistical learning, graph intelligence, and autonomous agents - indicating the emerging consensus that using graph-structured information along with agent-based cooperation is essential to solving the problems of the complex data flow management [9]. Another review paper prepared by Long et al. states that the newest generation of fault monitoring and diagnosis algorithms is now based entirely on machine learning and artificial intelligence as a response to the needs of engineering applications. Focus has changed to not just detecting an anomaly but integrating it as diagnosis, prediction, decision-making, and recovery [10]. To sum up, current sources of research can provide useful information about what is required to locate the roots and create repair solutions based on causal inference, although certain aspects such as dynamic updating of causal graphs, the choice of intervention strategies in case of safety constraints, as well as cross-scenario extrapolation still need optimization.

3. Methods in this paper

Consider the system multi-source observations as during a period of time, where $\mathbf{I}$ is the indicator vector (i.e., $\mathbf{I}$) and $\mathbf{S}$ is the tracking statistics (quantiles of the latency span, call failures, and so on), and $\mathbf{F}$ is the log/event features (frequencies of templates ID, abnormal words, etc.). Let us denote by $\mathbf{S}$ a service set and an indicator/event set by $\mathbf{I}$. The root cause factor may be service health or some particular indicator. Symptom set $\mathbf{S}$ is created by anomaly detection module (e.g., Latency $\uparrow$, ErrorRate $\uparrow$). We develop a two-layered SCM and couple this using mapping: the service level causal graph describes the calling and spreading, and the indicator level causal graph describes the causal interdependencies of the observable indicators. Its structural equation is: $\mathbf{I} = \mathbf{G}(\mathbf{S}, \mathbf{Z})$, where $\mathbf{S}$ is the set of parent nodes and $\mathbf{Z}$ is exogenous noise. The implementation of intervention is done through the do -operator: substitution of the equation as a constant z to represent injection and restoration interventions. To integrate both service and indicator levels, we use a service-indicator association matrix $\mathbf{A}$, and state that indicators/events are part of the services $\mathbf{S}$. Inference process anomalies are projected in the indicator layer via $\mathbf{A}$, and significant indicators of the indicator layer can also be traced back to the service nodes and provide coherent outputs on both coarse and fine-grained levels.

4. Experimental Setup

The paper has three test sets (Sock Shop, Online Boutique, Train Ticket) to cover 11 types of anomalies and 735 test cases [2]. A multivariate telemetry and service/metric indicator with multiple root causes are available in each of the test case lines. The perturbation injection and reproducibility tests are implemented in a Kubernetes cluster, which implements Chaos Mesh app to orchestrate the network latency, CPU consumption, misconfigurations, failures of dependencies, etc. [4].

Comparison Methods: Representative RCA algorithms, i.e., MicroRCA (propagation/graphology tracing), CausalRCA (causal structural learning), CoE (reasoning about event chains), and two variants of dynamic/adaptive causation models [1,3,5-6] were chosen. Self-healing comparisons utilized Rule (expert rules), Corr-Action (relevance-based action recommendation) and CIRCA-SH model discussed in this article.

Anomaly detection is implemented based on quantile threshold and seasonal decomposition to have low latency. Causal analysis window length $T_w = 5 - 10$ min, with a step size of 1 min, number of candidate root causes $K = 10$, and the top 3 are displayed. Self-healing execution action budget B specifies that only two actions may be taken to heal a given failure, and risk threshold p depends on the lowest possible rollback rate (e.g., $\leq 5\%$).

Statistical significance: To each sample, determine the indicator gap (Ours-baseline) and apply a paired-samples t-test; in the case of samples, do 1000 Bootstrap samplings to get the 95% confidence interval. The report includes both the p-value and the confidence interval in an attempt to avoid the reporting phenomenon of significance without effect size indication.

The table shows the minimum, as well as recommended requirements of both the computational resources and software stacks. Owing to the high volume of data read/write operations, batch training is relatively much more dependent on the CPU and memory. It has major applications in learning dynamic causal weights and evaluating the counterfactual outcomes. Simpler maintenance nodes are susceptible to online inference and decision-making and thus the crucial element here is the capability to get information at a fast rate and have steady and efficient telemetry lines. The suggested setup comprises a distributed storage/retrieval subsystem (like Thanos or ClickHouse) as it allows accommodating a broader variety of time intervals and more frequent measurements. OpenTelemetry is also

advised as the single tracking and acquisition tool to minimize the number of monitoring points needed when cross-programming languages are used. Offline replay testing and grayscale verification is the use of the chaos engineering component that must not be initiated regularly during peak business hours.

Table 1. System and software requirements

Component	Option	Minimum	Recommended
CPU	x86_64	8 cores	16+ cores
Memory	DDR4/DDR5	16 GB	64 GB
Kubernetes	v1.26+	v1.26	v1.28+
Metrics	Prometheus	v2.4+	v2.4+ + Thanos
Tracing	OpenTelemetry	Collector	Collector + Tempo/Jaeger
Logs	ELK / ClickHouse	ELK	ClickHouse + vector
Fault injection	Chaos Mesh	v2.5+	v2.6+

5. Results and Discussion

5.1. Quantitative Results and Visualization (Mandatory)

The interval-type uncertainty of the two major indicators is illustrated by error bars, Figure 1, with Precision@3 and MRR values presented as point estimates and 95% CI, respectively. In contrast to bar charts, this diagram clearly depicts whether it can be stated that the difference between methods is outside of the acceptable area of the random error variation between methods and is consistent with the findings of paired T-test/bootstraps. The CIRCA-SH approach, with a moderately stronger baseline, produces intervals of each indicator with some intervals shifting more to the right than others, with most intervals intersecting only around their right end, and indicates substantial changes. Moreover, this graphic can help to apply the structure of the effect size + confidence interval within articles, which is an appropriate method used by the best journals to report reproducible and reliable research findings.

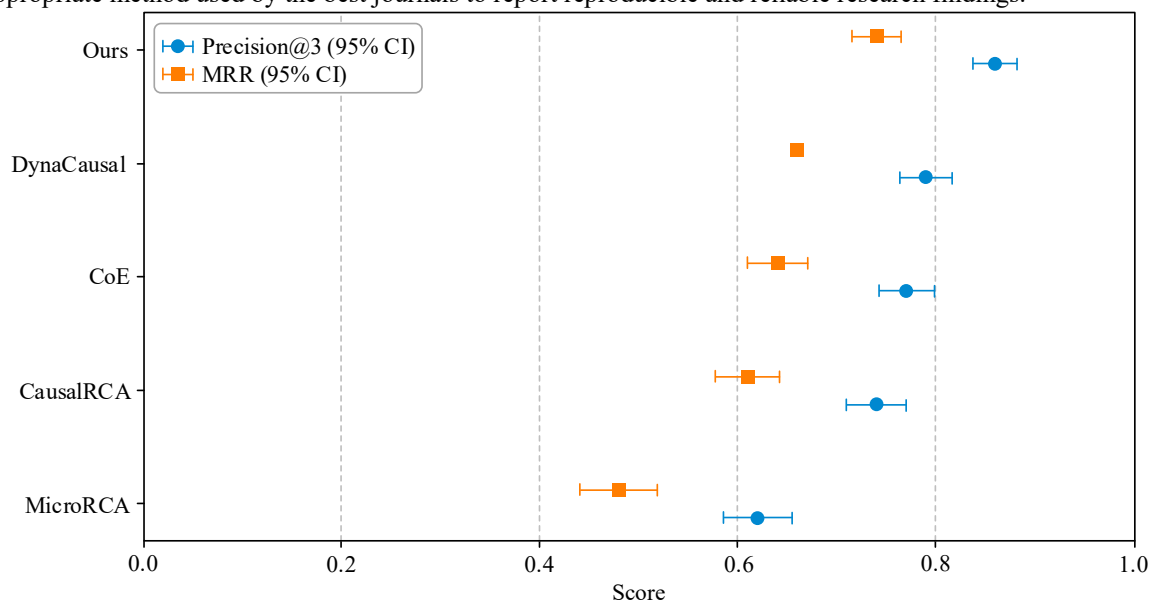


Figure 1. Point estimates of positioning indicators and their 95% confidence intervals (Forest-style).

5.2. Comparison Table of Latest Methods (Mandatory)

In Table 2 is contrasted between the Top-1 and Top-3 decision, MRR and single-event time in various popular approaches at constant RCAEval base line. The inclusion of causal inference (Causal Rca, COE, Dyna Causal, etc.)

usually enhances the localization accuracy, however, raises the cost of modeling and inference. CIRCA-SH maintains interpretability yet minimizes computational use by breaking down the SCM and updating it through a sliding window strategy. It runs comparably with strong baselines but performs more accurately indicating that the counterfactual evaluation approach combined with dynamic weighting can provide a better balance between efficient and effective work of the engineering system

Table 2. State-of-the-art comparison (RCAEval)

Method	Year	Dataset	Top-1	Top-3	MRR	Runtime(s)
MicroRCA	2023*	RCAEval	0.52	0.71	0.48	6.2
CausalRCA	2023	RCAEval	0.63	0.79	0.61	7.5
CoE	2024	RCAEval	0.65	0.81	0.64	8.1
DynaCausal	2025	RCAEval	0.66	0.82	0.66	9.5
Adaptive RCL	2025	RCAEval	0.67	0.83	0.67	9.2
CIRCA-SH (Ours)	2026	RCAEval	0.71	0.86	0.74	7.0

5.3. Qualitative Case Analysis (Mandatory)

As can be seen in Figure 2, the lollipop graph is indicative of the counterfactual level of optimization (- Score) of the purported roots causes. When compared to a histogram, this graph demeans the numerical figure and increases the ranking plus the difference and is able to minimize visual congestion so that it can be used to show the results of Top-K inferences. It is evident in the graph that Service B and DBCPU have the greatest disparities, and hence it is more probable that the treatment of these two indicators will rectify the syndrome. Although the network performance may be correlated, the discrepancy is lower, which may indicate that this is a secondary cause or ingress noise. Based on this, there is a direction of self-healing operations: focusing on DB-related capacity expansion or adding connection pools and applying rate limiting or ServiceB version rollback, which are better than the impulsive restarting of the whole system that can lower unnecessary rollback probabilities and shorten MTTR.

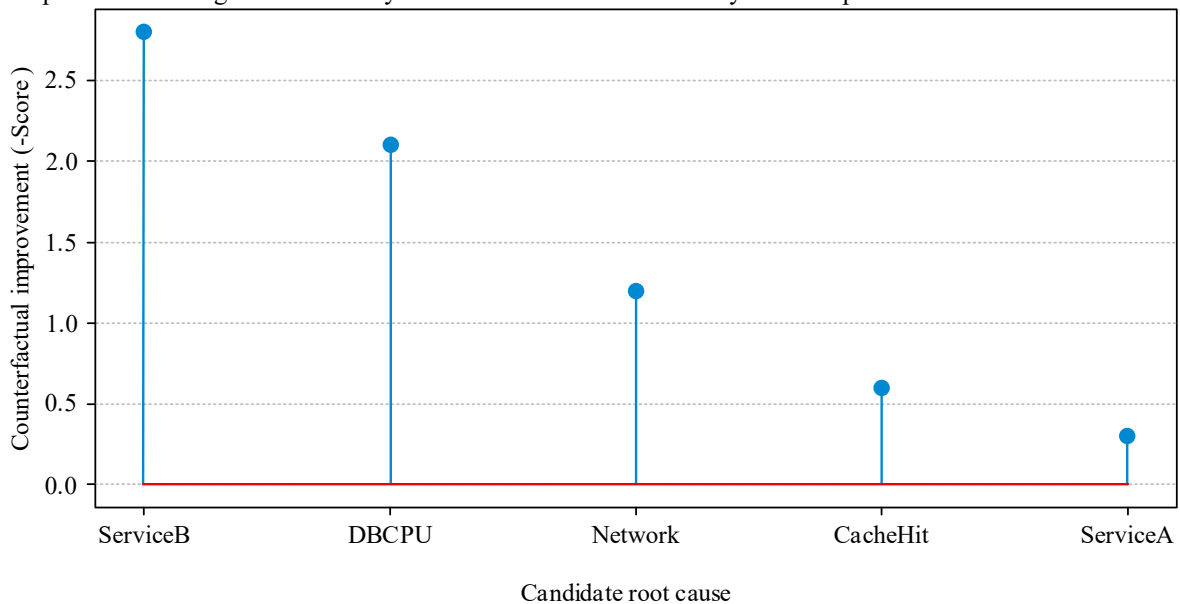


Figure 2. The best improvement level of the five highest alternative candidates who are counterfactual to a single case (Lollipop)

5.4. Self-healing effects, impacts, and insights

Violin plot is used to illustrate in Figure 3 the distribution form of the term the inference delay per incident (with the mean/median value) under varying service sizes. Unlike a trend line plot, which would only display the trend, the violon plot may reveal the tail risk: the schemes of the base line diverge widely with growth of scale, so that the inference cost variation rises; conversely, CIRCA-SH is quite narrow, which suggests that the techniques based on subgraph pruning and sparsification do not only decrease the average latency but also limit the likelihood of exceptionally slow cases. The current way of presenting this data is more compliant with SLO perspective in engineering (paying attention to P95 / P99 risk) and aligns better with the format of advanced statistical charts plus uncertainty in wording.

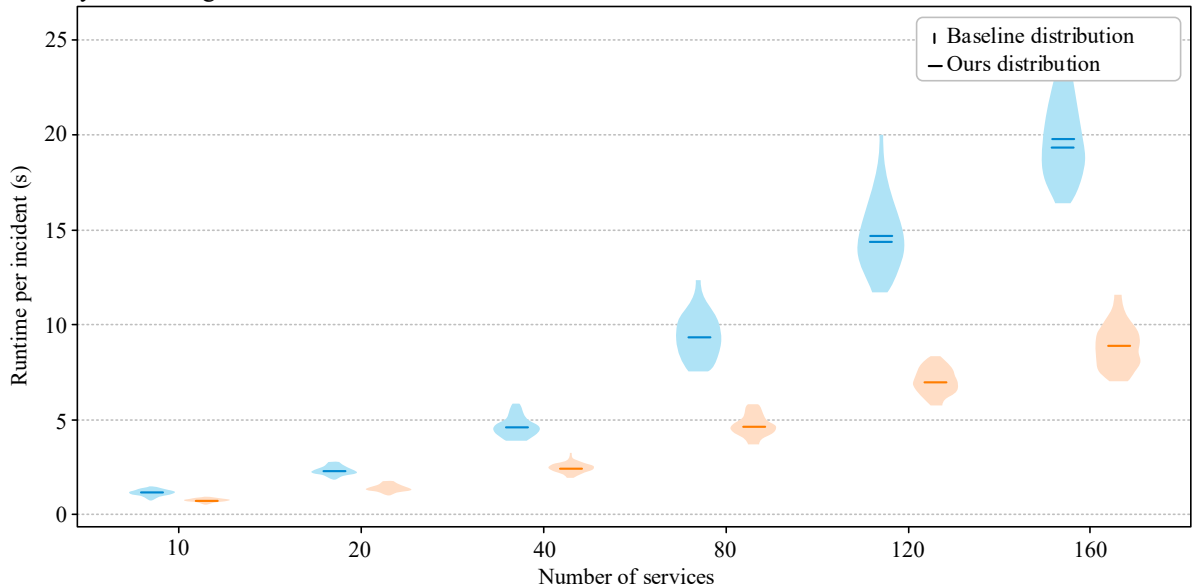


Figure 3. Scalability: Runtime distribution under different service scales (Violin)

5.5. Scalability and Universality

The Figure 4 shows intra-domain and cross -domain findings of each dataset in pairs of lines. The accuracy level will be determined by the position of the points, whereas the severity of the degradation will be reflected through the length of the line segments. This is more appropriate to be used in discussing cross-domain generalization compared to bar charts that are capable of presenting both absolute values and difference values on a single visual channel. It demonstrates the fact that accuracy reduced in all three systems but with a fairly equivalent amount of degradation which proves that CIRCA-SH is sensitive to structural causal relationships rather than scale changes. Moreover, this graph is useful in determining the most vulnerable datasets to degradation in the analysis of generalization, which reveals the hidden causes or origins of semantic inconsistencies.

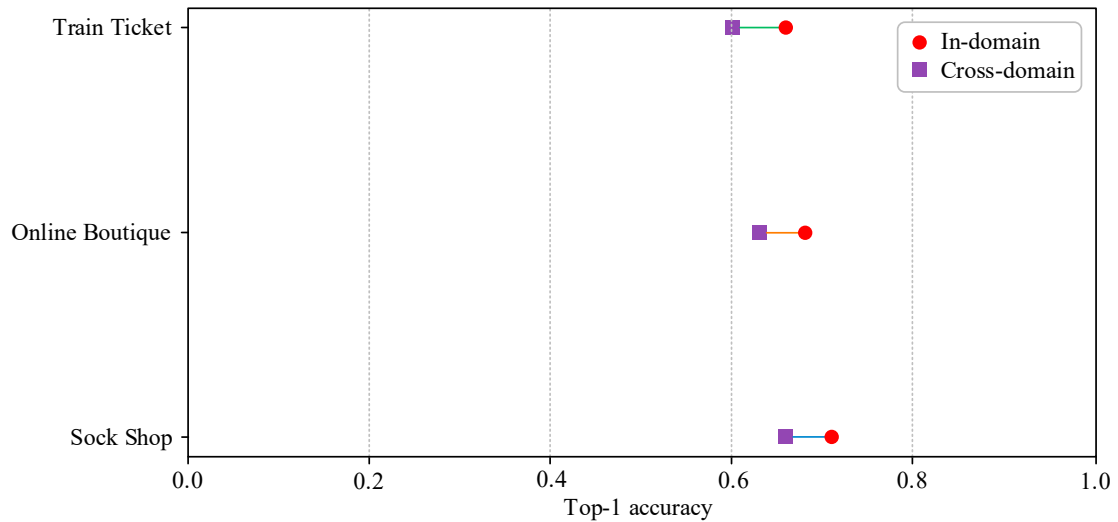


Figure 4. Usability: Comparison of Top-1 accuracy matching within and across domains (Dumbbell)

6. Conclusion

The present paper is going to design the CIRCA-SH framework which employs causal reasoning to identify the main causes behind failures of distributed systems and to derive risk-limited self-repairs strategies by predicting counterfactual actions and creating a closed loop feedback system of diagnosis, repair, testing and training. Empirical analyses indicate that, according to RCAEval criteria, this scheme performs better than many other existing state-of-the-art algorithms regarding different measures like Precision@3 and MRR, as well as is much lower in MTTR in self-healing scenarios. The future work will be devoted to learning-oriented causal exploration of latent variables and unseen observations, real-time intervention choices with secure causal reinforcement learning, and close integration with change management systems/rolling releases to make it even more effective and suitable to be scaled up and implemented under realistic operating conditions.

References

- [1] Chinnaiiah M, Kaata S K, Kohli R. Self-Healing Data Systems: AI-Augmented Root Cause Analysis in Modern Warehouses[C]//2025 2nd Asian Conference on Intelligent Technologies (ACOIT). IEEE, 2025: 1-7.
- [2] Rawas S, Samala A D, Fortuna A. CRISP: Cloud resilient infrastructure for self-healing platforms in dynamic adaptation[J]. International Journal of Information Technology, 2024: 1-14.
- [3] B T. Tutuncuoglu AI-Augmented Root Cause Analysis Autonomous Inference Engine of Multi-Layered System Failure [J]. To be published in SSRN 5343033, 2025.
- [4] Yi J, Hui Z, Wei Y, et al. CloudHeal is a Lightweight Online Learning-based Self-healing System in Cloud-native Systems[C], Proc. 25th Annual International Symposium on Reliable Distributed Systems (SRDS). Piscataway, NJ, USA: IEEE, 2025. 01-12.
- [5] Ji Y., He Z., Yang W. and others. CloudHeal: A Lightweight Online Learning Derived Self-Healing Framework of Cloud-Native Systems[C]//2025 44rd International Symposium on Reliable Distributed Systems (SRDS). IEEE, 2025: 01-12.
- [6] Fang T, Zhou Q, Brooks E. Multi-Granularity Dependency Modeling for Automated Fault Triage in High-Throughput Financial Transaction Systems[J]. Frontiers in Business and Finance, 2026, 3(1): 74-87.
- [7] Darshan S. Multi-Agent Orchestration for Autonomous Data Pipelines: A Systems Architecture for Self-Healing, Context-Aware, and Resilient Data Processing[J]. International Journal of Emerging Research in Engineering and Technology, 2026, 7(1): 47-53.
- [8] S N Rajwalasimha, D K J B Saini, S Kolekar, et al. Reliable and Self-Adapting AI in Cyber-Physical Systems That Heal Themselves: A Federated and Explainable Learning Approach[C]//2025 International Conference on Intelligent Computing, Information and Control Systems (ICOIICS) . IEEE, 2025: 1426-1433.
- [9] Somayajula R. Hybrid AIOps as an Enabler of Resilient Data Pipelines: The combination of statistical analysis, graph intelligence, and autonomous agents in order to proactively maintain them[C]//2025 IEEE 6th Global Conference on Improvement of Technology (GCAT). IEEE, 2025: 1-8.

- [10] Long S, Tan J, Mao B, et al. A survey on intelligent network operations and performance optimization based on large language models[J]. IEEE Communications Surveys & Tutorials, 2025, 27(6): 3915-3949.